

Slide 1

Dobrý deň, vážená komisia,
volám sa Andrii Stoliar a témou mojej diplomovej práce je **Návrh implementácie vybraných algoritmov riadenia**.

Práca bola vypracovaná na Ústave robotiky a kybernetiky pod vedením Ing. Mariána Tárnika, PhD.

Slide 2

Ciele práce boli zamerané na praktickú implementáciu riadiacich algoritmov na reálnom laboratórnom systéme.

Najskôr bolo potrebné vybrať laboratórny dynamický systém, oboznámiť sa s hardvérom a pripraviť krátku technickú dokumentáciu.

Ďalej bolo potrebné pripraviť komunikáciu pre tri platformy: MATLAB/Simulink, Arduino UNO a PLC Siemens.

Následne boli vybrané, opísané a implementované tri algoritmy: PI regulátor, PI regulátor s prepínaním pracovných bodov a adaptívny regulátor typu MRAC.

Hlavným cieľom bolo overiť, či je možné tieto algoritmy preniesť z návrhu do reálnej implementácie a experimentálne ich otestovať.

Slide 3

Na tomto slide je použité laboratórne zariadenie TS05.

Ide o tepelný systém s dvoma vstupmi a dvoma výstupmi. Vstupy sú napätie pre ventilátor a napätie pre vyhrievaciu špirálu, obidve v rozsahu 0 až 10 V.

Výstupy sú signály z dvoch teplotných senzorov, tiež v rozsahu 0 až 10 V.

Pre ďalšiu reguláciu bol systém uvažovaný ako SISO systém. Riadiacim vstupom bolo napätie na špirále, ventilátor bol konštantný a ako výstup bol použitý senzor 1.

Slide 4

Tu sú ukázané statické vlastnosti systému.

Statické charakteristiky boli merané pre tri pracovné body ventilátora: 3 V, 6 V a 9 V.

Pri každom meraní sa menilo napätie na špirále od 0 do 10 V a sledovala sa odozva oboch senzorov.

Ako príklad je tu zobrazený pracovný bod ventilátora 6 V. Práve táto hodnota bola použitá vo všetkých ďalších experimentoch.

Slide 5

Na identifikáciu systému boli vykonané merania skokových zmien v okolí jedného pracovného bodu. Ventilátor bol nastavený na 6 V, špirála na 4 V a zmeny vstupu boli $\pm 0,5$ V.

Z dát boli vytvorené modely dvoma metódami. Prvá bola metóda najmenších štvorcov a druhá bola optimalizácia parametrov prenosovej funkcie pomocou fminsearch.

Po prevode modelov do spojitého tvaru boli v čitateli veľmi malé koeficienty. Tieto členy boli zanedbané a aproximácia bola znovu overená.

Kedže sa RMSE zmenilo iba minimálne, ďalej bol použitý jednoduchší tvar prenosovej funkcie.

Slide 6

Tu je zhrnutá komunikácia so systémom TS05 pre jednotlivé platformy.

V MATLAB/Simulink bola použitá meracia karta Advantech a pripravené analógové vstupy a výstupy. Stavové premenné boli riešené cez Unit Delay bloky.

Pre Arduino UNO bolo potrebné vytvoriť vlastnú hardvérovú komunikáciu. Boli identifikované piny konektora, výstupy boli riešené cez PWM/0-10 V prevodník a signály zo snímačov cez napäťový delič. Použitá bola aj kalibrácia signálových ciest.

Pri PLC Siemens bol systém pripojený priamo cez analógový modul. Riadiaca logika bežala v bloku Cyclic Interrupt s periódou 0,1 s.

Slide 7

Prvým algoritmom bol PI regulátor pre riadenie v okolí jedného pracovného bodu.

Bol zvolený ako najjednoduchší prípad, na ktorom sa dá dobre ukázať celý postup návrhu a implementácie.

Regulátor pracuje s regulačnou odchýlkou, teda s rozdielom medzi žiadanou hodnotou a výstupom systému.

V diskretnom čase bolo potrebné počítat chybu, aktualizovať integrátor a vypočítat akčný zásah.

Parametre boli navrhnuté pomocou funkcie pidtune. Výsledné hodnoty boli $K_p = 3,58$ a $K_i = 1,84$.

Slide 8

Na tomto slide je ukázaný všeobecný postup implementácie v MATLAB/Simulink.

Najskôr bol algoritmus zapísaný ako pseudokód. Potom bola rovnaká logika prepísaná do bloku MATLAB Function a zapojená do Simulinku.

Kedže MATLAB Function blok sám neuchováva stavy medzi krokmi, integrátor a pracovný bod boli vedené spätnou väzbou cez Unit Delay bloky.

Slide 9

Ďalšia implementácia bola v jazyku C++ na Arduino UNO.

Tu bolo potrebné explicitne inicializovať premenné a priradiť im počiatočné hodnoty.

Regulačná logika bola podobná ako v MATLABe. Najskôr sa určil pracovný bod, potom sa počítala chyba, integrátor a akčný zásah.

Rozdiel bol v tom, že v C++ sa nepoužívala iba konštanta T_s , ale reálne nameraná dĺžka aktuálneho kroku programu.

Pri zložitejších algoritmoch boli doplnené aj pomocné funkcie pre prácu s vektormi a maticami.

Slide 10

Posledná implementácia PI regulátora bola realizovaná na PLC Siemens v TIA Portal.

Regulátor bol vytvorený ako Function Block. Výhodou je, že tento blok má vlastnú vnútornú pamäť, napríklad pre stav integrátora.

Premenné boli definované a inicializované v tabuľkovej časti bloku.

Blok mal dva režimy. V neaktívnom režime pripravoval pracovný bod a vnútorné premenné. Po aktivácii vykonával PI regulačný zákon.

Na obrázku je zapojenie v Cyclic Interrupt bloku. Je tam časovač, PI regulátor a blok pre logovanie dát.

Rovnaký princíp bol použitý aj pri ďalších algoritmoch. Podrobne ho ukazujem práve tu, lebo PI regulátor je najprehľadnejší.

Slide 11

Na tomto slide je porovnanie PI regulátora na troch platformách: MATLAB/Simulink, Arduino C++ a PLC Siemens.

Výsledky sú veľmi podobné, čo potvrdzuje, že rovnaký algoritmus bol úspešne implementovaný na všetkých platformách.

Rozdiely sú malé aj podľa hodnôt RMSE.

Pri PLC je priebeh menej zašumený, pravdepodobne vďaka spracovaniu analógového signálu a potlačeniu rušenia priamo v PLC module.

Slide 12

Ďalšou časťou bol riadiaci systém s prepínaním pracovných bodov.

Postup identifikácie a návrhu PI regulátora bol zopakovaný ešte pre ďalšie dva pracovné body. Celkovo boli použité tri body: 2 V, 4 V a 6 V na špirále.

Štruktúra má dve časti. Globálny PI regulátor zabezpečuje prechod medzi pracovnými bodmi a lokálny PI regulátor riadi systém v okolí aktuálneho bodu.

Globálna žiadaná hodnota je filtrovaná filtrom druhého rádu, aby prechod nebol príliš prudký.

Slide 13

Tu je matematická logika prepínacieho algoritmu.

Lokálny regulátor pracuje s odchýlkami od pracovného bodu, teda s Δy a Δu . Jeho parametre sa vyberajú podľa aktuálneho pracovného bodu, čo je princíp gain scheduling.

Globálny regulátor sa aktivuje pri zmene globálnej žiadanej hodnoty. Kým je aktívny, lokálny regulátor nepracuje.

Aby pri prepnutí nevznikol skok v akčnom zásahu, počiatočné hodnoty integrátorov sa dopočítavajú podľa aktuálneho výstupu regulátora.

Globálna regulácia sa ukončí po detekcii ustálenia. Podmienka je, že aspoň 25 z posledných 30 hodnôt výstupu je v pásme $\pm 3\%$ okolo globálnej žiadanej hodnoty.

Slide 14

Tu je príklad výsledku pri implementácii v PLC Siemens.

V grafoch sú výstupy, žiadané hodnoty, akčný zásah, zložky riadenia, parametre lokálneho regulátora a aktivačný signál globálnej vetvy.

Pri zmene globálnej žiadanej hodnoty sa aktivuje globálny regulátor. Po ustálení riadenie preberie lokálny regulátor.

Na detaile je vidieť tolerančné pásmo $\pm 3\%$. Ako nový pracovný bod sa nepoužije priamo žiadaná hodnota, ale reálna hodnota získaná ako priemer posledných 30 vzoriek pred prepnutím.

Slide 15

Na tomto slide je porovnanie prepínacieho algoritmu na všetkých troch platformách.

Priebehy výstupu aj akčného zásahu sú veľmi podobné.

To potvrdzuje, že algoritmus bol úspešne implementovaný v MATLAB/Simulink, v C++ na Arduino UNO aj v PLC Siemens.

Slide 16

Tretím algoritmom bolo adaptívne riadenie s referenčným modelom, teda MRAC.

Cieľom je, aby výstup reálneho systému sledoval výstup referenčného modelu.

Regulácia sa realizuje v súradniciach pracovného bodu, preto sa používajú odchýlkové veličiny Δy , Δr a Δu .

Riadiaci zákon je daný súčinom vektora parametrov Θ a regresného vektora ω .

Parametre Θ sa počas behu menia podľa adaptačného zákona. Používa sa aj numerická derivácia výstupu, preto je algoritmus citlivejší na šum merania.

Slide 17

Pri MRAC regulátore bolo vhodné ukázať výsledky platforiem samostatne.

Vľavo sú výsledky z MATLAB/Simulink. Tu je sledovanie referenčného modelu veľmi dobré a parametre Θ sú pomerne stabilné.

Vpravo sú výsledky z PLC Siemens. Algoritmus funguje aj tu, ale priebeh nie je taký hladký ako v MATLABe.

Parametre Θ pri PLC mierne kolíšu, čo môže súvisieť so spracovaním signálov, časovaním a vlastnosťami platformy.

Slide 18

Pri implementácii MRAC v C++ sa ukázal výraznejší drift adaptačných parametrov.

Preto bola navrhnutá jednoduchá heuristická modifikácia. Nešlo o zmenu princípu MRAC, ale o praktickú úpravu implementácie.

Úprava spočívala vo vyhladení meraného výstupu a v zavedení deadzone pre chybu sledovania. Pri malej chybe sa adaptácia parametrov obmedzila, a tým sa potlačil drift.

Na výsledkoch je vidieť, že modifikácia pomohla. Akčný zásah je menej zašumený a drift parametrov je výrazne menší.

Adaptačné koeficienty α boli na všetkých platformách rovnaké a boli zvolené empiricky podľa MATLAB implementácie.

Detailnejšie príčiny rozdielnej dynamiky môžu byť predmetom diskusie po prezentácii.

Slide 19

Na záver môžem zhrnúť, že v práci bol zdokumentovaný systém TS05 a pripravená komunikácia pre tri platformy.

Boli implementované tri algoritmy riadenia: PI regulátor, PI regulátor s prepínaním pracovných bodov a MRAC regulátor.

Výsledky ukázali, že algoritmy je možné úspešne spustiť na reálnom systéme.

Pri MRAC regulátore sa ukázalo, že aj pri rovnakej logike implementácie môžu rôzne platformy viesť k rozdielnym výsledkom.

Slide 20

Ďakujem za pozornosť.

Otazka 1

Najjednoduchšie riešenie by bolo doplniť logovací blok o resetovací vstup. Pri nábežnej hrane resetovacieho signálu by sa vynuloval index, príznak Full a hodnota SampleCount. Samotné polia by nebolo nutné vždy fyzicky nulovať, pretože za platné by sa považovali iba vzorky od začiatku po aktuálnu hodnotu SampleCount. Staré údaje za týmto rozsahom by sa jednoducho ignorovali. Ak by však bolo potrebné zabrániť omylu pri manuálnom kopírovaní dát, bolo by možné v resetovacej fáze polia aj explicitne vyčistiť.

Praktickejšie riešenie by bolo spracovať logger ako samostatný stavový automat. Ten by mohol mať napríklad stavy IDLE, RESET, MEASURE a DONE. V stave IDLE by logger čakal na príkaz na nové meranie a nezapisoval by dáta. Po prijatí štartovacieho signálu by prešiel do stavu RESET, kde by sa pripravili všetky vnútorné premenné, teda index, SampleCount a príznak Full. Potom by nasledoval stav MEASURE, v ktorom by sa pri každej vzorke zapisovali aktuálne hodnoty do polí. Po naplnení bufferu alebo po ukončení merania by logger prešiel do stavu DONE, kde by boli údaje pripravené na export alebo kopírovanie. Takéto riešenie by umožnilo opakovane spúšťať nové merania bez zmeny režimu CPU a zároveň by bola logika merania prehľadnejšia.

Pre kontinuálne merania by som použil kruhový buffer. V takom prípade by sa po naplnení poľa nezačalo logovanie zastavovať, ale najstaršie vzorky by sa postupne prepisovali novými. Logger by teda stále uchovával posledný časový úsek merania. Ak by operátor potreboval daný priebeh uložiť, mohol by sa na základe signálu aktuálny obsah dynamického bufferu skopírovať do jedného z viacerých pamäťových bufferov ako samostatný záznam merania. Takéto riešenie by bolo vhodné najmä vtedy, keď meranie prebieha dlhodobo a dôležité udalosti chceme uložiť až spätne po ich výskyte.

Otazka 2

Áno, takéto porovnanie by bolo možné hlavne pre základný PI regulátor na PLC.

PID_Compact používa inú parametrizáciu. Ak derivačnú časť vypneme nastavením $T_d = 0$ a nastavíme $b = 1$, dostaneme PI regulátor. Potom možno parametre prepočítať ako $K_p_compact = K_p$ a $T_i_compact = K_p / K_i$.

Teoreticky by preto mala byť dynamika veľmi podobná. Nemusí však byť úplne rovnaká, hlavne pri saturácii, pretože PID_Compact obsahuje aj vnútorné mechanizmy ako anti-windup, režimy práce a diagnostiku. Moja implementácia mala jednoduchšiu saturáciu a bola priamo viditeľná v kóde.

Výhodou PID_Compact je, že ide o hotový priemyselný blok s autotuningom a diagnostikou. Nevýhodou z pohľadu tejto práce je menšia transparentnosť, väčšia univerzálnosť bloku a menšia flexibilita pri úpravách algoritmu.

Vlastný blok bol jednoduchší, výpočtovo ľahší a dalo sa ho ľahko preniesť medzi MATLAB, C++ a PLC. Navyše parametre K_p a K_i boli nezávislé. Preto by napríklad čistý I regulátor bolo možné realizovať jednoducho nastavením $K_p = 0$ a $K_i \neq 0$. Pri parametrizácii PID_Compact je integračná časť viazaná na K_p cez T_i , takže takéto riešenie nie je také prirodzené.

Preto by PID_Compact bol vhodný ako referenčné priemyselné riešenie, ale pre cieľ mojej práce bola vhodnejšia vlastná transparentná implementácia.